

FULL STACK DEVELOPMENT

FASE 5 | AULA 05 -

CLEAN ARCHITECTURE - PROGRAM PARADIGMS E COMPONENTS PARADIGMS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMSE

O QUE VEM POR AÍ?

Depois de discutirmos sobre os casos de usos, vamos subir mais um degrau em nossa camada da Arquitetura Limpa. Falaremos agora sobre Controllers, Gateway e Presenters. Essas camadas que fazem o meio de campo entre os Devices, APIs e bancos de dados, são essenciais para o funcionamento das nossas soluções e para adequar os dados para serem utilizados nos casos de usos e entidades.



HANDS ON

As duas camadas que discutiremos aqui são as camadas que fazem a interface com o “mundo real”. A Arquitetura Limpa permite que o isolamento funcione de modo segregado, e garantindo que a regra de negócio não esteja misturada com os controles de operação do software e com os serviços externos.

Discutiremos isso nos vídeos a seguir, onde falaremos sobre o papel dessas camadas no processo de construção do software e como essa segregação é vital para o sucesso do software que estamos implementando.

Para essa aula, temos um link para você poder treinar o conteúdo ensinado. Acesse:

[Base de dados da aula 05.](#)

SAIBA MAIS

Por que estas camadas existem?

Até o momento, nossa maior preocupação era construir, de forma robusta, o funcionamento do software pensando apenas no **domínio**, ou seja, nas funcionalidades que os clientes esperam que o software atenda. Até agora, isso atende ao esperado, mas o software precisa se comunicar com o mundo exterior para que seja usado. E para fazer essa comunicação com o mundo exterior, precisamos implementar camadas externas, ao redor das camadas de regras de negócio.

As duas camadas usadas têm diferentes responsabilidades, e quanto mais "externas" no desenho, mais elas estarão perto do usuário e da comunicação com o mundo exterior.

Quais são as camadas externas?

Adaptadores de Interface

A primeira camada é a de adaptadores de interface, que é responsável por fazer a intermediação entre os elementos externos e internos. É a camada de adaptadores de interface que recebe as chamadas vindas de uma API e faz a chamada de um caso de uso para fazer com que o software execute a funcionalidade. Nessa camada também fazemos dois trabalhos importantes para que a comunicação ocorra sem problemas: adaptar os dados que as camadas externas informam, e devolver corretamente os dados para quem requisitou.

Nessa camada de adaptadores de interface, fazemos o software realizar um papel semelhante ao de um balconista de lanchonete ou de uma pessoa que trabalha em uma loja de roupas. No caso da lanchonete, pegamos o pedido, adaptamos para que a cozinha execute e entregamos para o cliente o pedido para o consumo. Já no caso da loja de roupas, o vendedor entende o que você precisa e busca algo adaptado à sua necessidade.

No primeiro caso, quando você chega em uma lanchonete, já existe uma lista de produtos à venda, e você deve escolher um deles. No máximo, poderá pedir a alteração em alguns, mas algo que não altere demais os processos de produção dos

sanduíches que já existem. E, ao final, o pedido vem também de uma forma padrão, já esperada por você. Se pensarmos que nossos casos de uso trabalham com os dados das entidades, e são regras codificadas, o balcão serve apenas como um ponto de processamento entre duas camadas.

No caso da loja de roupas, existe um diferencial: você pede uma roupa e a pessoa que trabalha na loja vai precisar entender o que, entre as opções disponíveis, te atende. A partir daí, ela decide o que oferecer. Do mesmo jeito, quando o processamento entre um Cliente de nossa aplicação pede a execução de uma regra de negócio, ele pode não informar os dados com o mesmo formato que nosso caso de uso espera, ou pode esperar que o retorno seja num formato diferente do que ele informou: por isso, precisamos que o ponto de processamento da comunicação entre as duas camadas também entenda como ajustar os dados para uso, e como adaptar a resposta para cada Cliente.

Frameworks & Drivers

A camada mais externa é a de Frameworks e Drivers, e é a que representa o mundo exterior. É nesta camada que usamos as tecnologias, bibliotecas e funções que se comunicam com o mundo real.

É nesta camada que fazemos, finalmente, a implementação das rotas de nossa API, criamos o servidor e fazemos o tratamento dos dados que cada cliente (seja um software ou uma pessoa) informa. É aqui que lidamos com a parte do sistema que não se envolve com as regras de negócio, e que foca mais nas regras de implementação, como a definição de como a API vai funcionar, quais as regras de segurança para os clientes, as camadas de autenticação e autorização, e assim por diante.

Aqui também ficam as interfaces com as origens de dados, sejam repositórios de dados, APIs externas e outras que o software usa para armazenar seu estado, comunicar com APIs e ferramentas externas, e tudo o que não faz parte de sua responsabilidade.

Esta é a camada mais externa da arquitetura, é a camada que os elementos externos veem. Ela é o suporte operacional do software e permite que o software funcione plenamente no mundo real. Se o nosso software fosse um computador, essa camada seria composta pela tela, pelo teclado, pelas portas de conexão: são

importantes para fazer o computador funcionar e desempenhar seu trabalho, mas não são parte do funcionamento interno do computador. Não é o teclado que executa o trabalho, mas sem ele é impossível informar ao computador o que deve ser feito.

Adaptadores de Interface

Como vimos anteriormente, a camada de Adaptadores de Interface (ou Interface Adapters) é responsável por fazer a comunicação entre a camada de interface com os clientes e a camada de casos de uso. Os clientes externos não devem acessar diretamente a camada de casos de Uso, porque todas as requisições devem ser adaptadas nos dois sentidos, na recepção e na entrega.

Os casos de uso precisam receber os dados no formato em que entendem, seja entidades, objetos de comunicação de dados, ou até mesmo dependências injetadas. Mas não é papel da camada externa informar os dados desse jeito; as camadas exteriores não têm a responsabilidade de preparar os dados e nem a camada de casos de uso tem a responsabilidade de adaptar os dados.

As camadas de adaptadores de interface servem também para padronizar o acesso aos casos de uso. Dessa forma, quando um cliente externo deseja acessar uma regra de negócio, ele acessa um componente dessa camada, e daí este componente se coordena com outros componentes dessa camada para fazer todo o trabalho de adaptação da requisição na entrada, a requisição ao caso de uso referente à funcionalidade, e a adaptação dos dados da saída.

Quais os tipos?

Controller

O primeiro tipo de componente que encontramos nesta camada é o **Controller**. O objetivo dos componentes deste tipo é o de servir como o conector entre as duas camadas (acima e abaixo) e, ao mesmo tempo, organizar os passos da requisição efetuada, instanciando e executando tudo o que for necessário para que o resultado seja entregue.

No caso de implementarmos um software para controle de uma universidade, os controllers são os elementos que fazem o trabalho de receber os dados de uma

chamada externa, como os dados de uma pessoa e a disciplina que esta pessoa quer cursar, transformar estes dados em entidades, chamar o caso de uso `MatricularAluno`, receber o retorno da execução e preparar o retorno no formato que o requisitante precisa.

Não é obrigação do Controller saber **como** o caso de uso funciona, só saber como o caso de uso precisa dos dados, e como o caso de uso vai retornar. Então, para o Controller não existe “sucesso” ou “erro”. Ele é apenas um “mensageiro” que faz a comunicação funcionar corretamente.

Presenters

Encontramos também os **Presenters**, que são componentes que servem para uma única finalidade: preparar os dados para o retorno ao cliente que acessou um Controller.

Este comportamento é segregado porque ele tende a ser complexo: não significa apenas retornar os dados, mas retornar no modelo que o cliente consegue entender. Por exemplo, se o cliente está usando uma API HTTP RESTful, o retorno tem que seguir os padrões HTTP, com os cabeçalhos corretos, o conteúdo formatado de acordo com o cabeçalho Content, e assim por diante. E esta responsabilidade não é do Controller que é chamado, ele não deve saber como devolver os dados, mas sim saber quem pode ajustar o retorno que ele precisa fazer.

Como vimos, usualmente um Presenter é instanciado por um Controller, porque não faz sentido um componente da camada exterior tentar formatar os dados da entrega. Por isso, os componentes desse tipo são usados apenas dentro da camada de Adapters.

Gateways

Os **Gateways** são os componentes que implementam a comunicação com os componentes externos que fornecem ou armazenam dados ou efetuam atividades externas ao software. São os componentes que fazem com que os casos de uso possam acessar os repositórios de dados, por exemplo.

A implementação dos Gateways sempre tende a criar uma forma intermediária entre o serviço externo e a camada de casos de uso. Isso porque cada Gateway deve

expor suas ações como uma atividade a executar, e não como uma função de baixo nível no serviço externo.

Por exemplo, se vamos construir um Gateway para acesso a dados do tipo Estudante, num repositório externo, precisamos expor este serviço no formato que o caso de uso entenda:

```
class EstudanteGateway {  
    obterEstudantePorPessoa(PessoaEntity): EstudanteEntity {  
        // ...  
    }  
}
```

Para o caso que uso que vai receber este Gateway via Injeção de Dependência, o trabalho é simples: apenas requisitar os dados pelo método; e para o Gateway também, porque ele sabe como falar com o serviço externo, como usar os dados da entidade para obter os dados solicitados, e pode também entregar para o caso de uso a entidade formatada.

Frameworks e Drivers

Nesta camada, usamos as ferramentas disponíveis na linguagem de programação desejada, seja via ferramentas já existentes na linguagem ou via pacotes que instalamos, para criar a comunicação com o mundo exterior.

São nos elementos dessa camada que criamos o acesso ao repositório de dados, que implementamos a camada de API que o software vai usar, e assim por diante.

Esta é a camada que é visível no software; é por ela que sabemos que o software funciona. Se estamos fazendo uma aplicação desktop, nessa camada definimos as telas, o funcionamento dos botões, o comportamento de cada seção da tela. E neste exemplo, implementamos nos botões as chamadas aos Controllers, como necessário.

Do mesmo modo, implementamos as conexões aos repositórios de dados que são injetadas ao chamar um Controller, as conexões aos serviços externos e seus controles de erro. Dessa forma, conseguimos isolar totalmente os erros que podem

acontecer ao trabalhar com estes elementos externos. Se, por exemplo, há um erro por conta de indisponibilidade de conexão a um serviço de dados, podemos isolar isso rapidamente, sem precisar ficar procurando pelo código onde esta conexão existe, onde estão os dados e etc.

O isolamento dessa camada externa, que em muitos artigos e livros também é chamada de camada de infraestrutura, permite a segregação entre as dependências técnicas do software, e permite também que novas interfaces sejam adicionadas ao software sem precisar alterar a estrutura de funcionalidades: implementamos um novo componente, e este é responsável por servir de comunicação entre o consumidor e o núcleo da aplicação.

Isso tem sido especialmente aplicado ao construir interfaces Backend-to-Frontend, onde criamos camadas de API (seja web ou não) adaptadas para um consumidor específico, ao invés de construir interfaces genéricas usadas por muitos consumidores diferentes entre si. Podemos construir três APIs diferentes, uma para atender navegadores web, outra para atender dispositivos Android e outra para atender dispositivos iOS. Cada uma dessas interfaces pode ser especializada para tratar as particularidades de cada cliente, e não precisamos complicar nossa interface com diferentes códigos que servem apenas para adaptar as entradas e respostas de acordo com o cliente.

O QUE VOCÊ VIU NESTA AULA?

As duas últimas camadas são as que fazem o software ser útil. Sem elas, não importa a qualidade de nossa implementação das regras de negócio, não conseguiremos usar essas regras.

Uma das camadas serve para coordenar e organizar a comunicação entre o mundo exterior e o software. A camada de adaptadores de interface abriga os componentes responsáveis por organizar e padronizar a execução dos casos de uso (Controllers), por adequar as respostas de acordo com o requisitante (Presenters), e por permitir o acesso aos dados disponíveis fora do software (Gateways).

A última camada é onde implementamos o acesso aos dados externos e onde implementamos os componentes do software que expõem os casos de uso para os clientes externos, muitas vezes usando frameworks e soluções comuns de software que nos ajudam a lidar com a responsabilidade de criar APIs e interfaces semelhantes.

O que achou do conteúdo? Conte-nos no Discord!

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

EVANS

PALAVRAS-CHAVE

Gateways. Presenters. Controllers.

EMEND

POS TECH